Proposal for Orbital 26

Members:

Bawa Dhruv Munish, A0323053U, e1525960@u.nus.edu
Tauzih Ul Rashid Khan, A0315613M, e1512564@u.nus.edu

Team Name:

MINDMESH

Proposed Level of Achievement:

ARTEMIS

Motivation

The current workflow of learning from assignments and practice papers is highly inefficient and cognitively taxing. Students often annotate directly on physical papers or maintain fragmented notes across multiple platforms, resulting in unstructured, non-searchable, and poorly interconnected knowledge.

More critically, there exists a fundamental disconnect between practice and knowledge consolidation. While assignments are one of the most effective mediums for deep learning, the insights derived from them are rarely captured in a structured and reusable format. This leads to repeated mistakes, loss of valuable learning moments, and an inability to identify patterns across topics.

In addition, transforming raw learning into organised notes requires significant manual effort. Students must not only interpret their mistakes, but also spend time reformatting rough thoughts into coherent notes, introducing unnecessary overhead that detracts from the learning process itself.

We identify this as a systems-level problem: there is no seamless pipeline that converts learning-by-doing into structured, interconnected knowledge. Existing tools also fail to establish meaningful relationships between concepts across different assignments, forcing students to manually discover links between topics.

Our project is motivated by the need to redesign this workflow. We aim to do so by minimising friction in knowledge capture, automating the structuring process, and systematically organising knowledge using topic-based metadata and relationships between concepts. This enables the construction of interconnected knowledge representations, allowing students to better understand how different topics relate to one another. Ultimately, this allows students to focus on learning itself rather than managing their notes.

Aim

Our project aims to design and develop an AI-assisted learning system that transforms unstructured learning from assignments and practice papers into structured, interconnected, and reusable knowledge.

Specifically, we aim to build an end-to-end pipeline that captures users' rough notes and problem-solving experiences, automatically processes them into well-formatted coursework flashcards, and organises them using meaningful metadata such as topics and related concepts. This structured representation enables efficient storage, retrieval, and reuse of learning insights while reducing the manual effort required for note-taking.

In addition, we aim to enable higher-level knowledge synthesis by aggregating individual learning points into topic-based summaries and systematically establishing relationships between concepts. These relationships will be derived from metadata and visualised through an interactive, graph-based mindmap interface, allowing users to intuitively explore how different topics are connected.

Building on this, we aim to augment these connections with on-demand AI-generated insights. By interacting with links between related topics, users will be able to generate contextual explanations and cross-topic insights, enabling a deeper understanding of how concepts intersect and may be applied together.

Ultimately, our goal is to minimise the cognitive overhead associated with managing notes and maximise the efficiency of learning from practice, enabling users to focus on understanding concepts rather than organising information.

User Stories

Users of our system will be able to:

1. Capture Learning from Practice
    - Upload questions from assignments or practice papers (e.g. via screenshots) with rough notes describing what they learned from each question
2. Generate Structured Flashcards Automatically
    - Convert rough notes into well-formatted, readable coursework flashcards (CFCs) flashcards using AI
    - Store flashcards with associated metadata (e.g. topic, related topics, coursework context)
3. Organise Knowledge by Topics
    - Automatically categorise flashcards based on topics and related concepts
    - Navigate and retrieve notes through a structured hierarchical system (e.g. course → coursework → flashcards)
4. Aggregate Learning into Topic-Level Summaries
    - View combined insights from multiple flashcards under the same topic through topical flashcards (TFCs)
    - Identify recurring mistakes and patterns across different assignments
5. Visualise Knowledge as an Interactive Mindmap
    - Explore topics through a graph-based interface
    - View relationships between different concepts as connected nodes derived from topic metadata and related topics
6. Generate Cross-Topic AI Insights
    - Interact with links between related topics
    - Trigger on-demand AI-generated insights that explain how concepts are connected or may be applied together
7. Review and Revise Efficiently
    - Study using generated flashcards for active recall
    - Revisit past learning in a structured and easily accessible format
8. Share Notes with Other Users
    - Share flashcards and topic-based notes with friends through a private network
    - Upload topical flashcards (TFCs) to a public platform for wider access
9. Discover and Import Shared Content

- Browse notes publicly shared by other users
- Import external flashcards into their own system (with user approval) and merge them into their existing knowledge database

Features

1. Feature 1 (core): AI-Generated Coursework Flashcards
    a. Users can upload questions from assignments or practice papers and input rough notes. The system will use AI to convert these inputs into well-structured, readable coursework flashcards (CFCs) that capture key learning points, along with associated metadata (e.g. topic, related topics, coursework context).

2. Feature 2 (core): Topic-Based Organisation and Metadata Tagging
    a. Each generated flashcard will be automatically tagged with topics and related concepts. This allows the system to organise flashcards in a structured hierarchical format (e.g. course → coursework → flashcards) and enables efficient retrieval and navigation of knowledge.

3. Feature 3 (core): Topical Flashcard Aggregation
    a. The system will aggregate multiple coursework flashcards into topic-based summaries (Topical Flashcards (TFCs)). This allows users to view consolidated insights and identify patterns or recurring mistakes across different assignments.

4. Feature 4 (core): Community Notes Hub
    a. The system will include a public platform where users can upload and share their topical flashcards (TFCs). Other users can browse, import, and merge these notes into their own database. (Incentive to upload notes into the Community Notes Hub is yet to be decided)

5. Feature 5 (extension): Private Sharing System
    a. Users will be able to share flashcards and notes with friends through a private network. This enables collaborative learning in a controlled, non-public environment.

6. Feature 6 (extension): Interactive Mindmap Visualisation
    a. Users will be able to explore their knowledge through an interactive, graph-based mindmap. Topics will be represented as nodes, with connections indicating relationships between concepts derived from metadata such as related topics, enabling intuitive navigation of interconnected knowledge.

7. Feature 7 (extension): AI-Generated Cross-Topic Insights
    a. Users can interact with connections between related topics in the mindmap to generate on-demand AI-powered insights. These insights will highlight relationships between concepts and provide deeper understanding of how topics may be applied together.

8. Feature 8 (extension): Rating and Search System
    a. Users will be able to search for notes based on subjects or topics, and evaluate shared content through ratings or upvotes. This helps surface higher-quality notes and improves discoverability.

Timeline

Milestone 1 - Technical proof of concept

(Minimal working system with end-to-end integration)

- Feature 1 (AI-Generated Coursework Flashcards) done to a basic generation level
    - Users can input rough notes and receive structured flashcards via AI
- Feature 2 (Topic-Based Organisation and Metadata Tagging) done to a basic tagging level
    - Flashcards are tagged with simple topics generated by AI
- Feature 1 & 2 integration done such that basic end-to-end pipeline present
    - Input → AI processing → flashcard output → display
- Storage system done to a basic persistence level
    - Flashcards and metadata stored locally (SQLite)
- Frontend-backend integration done to a basic functional level
    - React UI connected to Spring Boot backend and AI API

Milestone 2 – Prototype (Core System Complete)

(Working system with all core features)

- Feature 1 (AI-Generated Coursework Flashcards) done such that outputs refined generation
    - Improved formatting, consistency, and reliability of output.
- Feature 2 (Topic-Based Organisation and Metadata Tagging) Have a structured hierarchy set in place
    - Flashcards organised into course → coursework → topic structure
- Feature 3 (Topical Flashcard Aggregation) done to a level of functional aggregation
    - Multiple flashcards combined into topic-level summaries (TFCs)
- Feature 5 (Private Sharing System) done to a usable level
    - Users can share flashcards with friends through a controlled network
- Feature 4 (Community Notes Hub) done to such that basic sharing and import functionality present
    - Users can upload, browse, and import shared topical flashcards
- Authentication system done to level where basic account management exists
    - Users can create accounts and access shared features
- Backend infrastructure done to a production-ready structure
    - PostgreSQL integration for shared data
    - REST API endpoints stabilised

Milestone 3 – Extended System (Core + Extensions)

(Complete system with advanced features)

- Feature 6 (Interactive Mindmap Visualisation) done to a functional visualisation level
    - Topics displayed as nodes with connections based on metadata
- Feature 7 (AI-Generated Cross-Topic Insights) on-demand insight generation available
    - Users can generate AI insights from connections between topics

- Feature 8 (Rating and Search System) basic discoverability level
    - Users can search notes and evaluate content via ratings/upvotes

- Feature 4 & 5 (Sharing Systems) Improvements to enable enhanced usability
    - Improved UX for sharing, importing, and managing notes

- System performance and UI developed to produce a polished user experience
    - Optimised queries, smoother interactions, improved interface design

Tech Stack

1. Technology 1: **React**
    a. Used to build the frontend user interface, including the flashcard views, topic navigation system, and interactive mindmap components.

2. Technology 3: **Java (Spring Boot)**
    a. used to implement the backend services, business logic, AI integration, storage access, and sharing APIs.

3. Technology 4: **SQLite**
    a. Used for local storage of users' flashcards, metadata, and topic structures in a lightweight relational database.

4. Technology 5: **PostgreSQL**
    a. Used as the server-side database for user accounts, shared notes, marketplace content, ratings, and other online features

5. Technology 6: **OpenAI API**
    a. Used to process user inputs and generate structured coursework flashcards, extract metadata such as topics and related concepts, and produce on-demand cross-topic insights.

6. Technology 7: **Graph Visualisation Library (e.g. React Flow)**
    a. Used to implement the interactive mindmap interface for visualising relationships between topics and enabling intuitive navigation of interconnected knowledge.

7. Technology 8: **Git and GitHub**
    a. Used for version control, collaboration, and code management throughout the development process.

Qualifications

Both members of our team have prior experience in building software systems and working with modern development tools.

We participated in the NTU Deep Learning Hackathon 2026, where we developed an application aimed at helping students better understand and improve their learning. The system consisted of a Python-based backend and a JavaScript frontend, with communication handled through API endpoints. Through this project, we gained experience in integrating frontend and backend components, working with real-world data, and designing user-facing features that provide actionable insights.

In addition, both team members are familiar with version control using Git, including collaborative workflows such as branching, merging, and maintaining a clean commit history. This allows us to work effectively as a team and manage changes to the codebase in a structured manner.

We are also comfortable with Java and object-oriented programming, including the design of modular and maintainable code. One team member has experience with common software design patterns such as the Factory, Builder, and Strategy patterns, and has been exposed to broader system design concepts. This provides a foundation for implementing structured backend logic using frameworks such as Spring Boot.

Furthermore, one team member was involved in the Harvard CS50 course, which covers a wide range of foundational computer science topics including algorithms, data structures, memory management, web development, and software engineering principles. This experience contributes to a solid understanding of how to design and implement efficient and reliable systems.

Academically, our team has performed consistently in computing-related modules, including:

Tauzih: A- for CS2030 (Programming Methodology II), B+ for CS1010X

Dhruv: B+ for CS1101S

These experiences have equipped us with a strong foundation in programming, problem-solving, and software development practices, and we are committed to further developing our skills through the Orbital programme.

Software Engineering

We plan to apply several software engineering techniques and practices throughout the development of our project to ensure that the system remains modular, maintainable, and scalable as its complexity increases.

First, we will adopt a layered system architecture to separate concerns across the application. The frontend will handle user interaction and visualisation, while the backend, implemented using Spring Boot, will manage business logic, data processing, and communication with external AI services. Within the backend, we will organise functionality such as flashcard generation, metadata extraction, topic aggregation, and sharing features into separate components to reduce coupling and improve maintainability.

Second, we will follow an incremental, milestone-driven development approach aligned with Orbital's evaluation structure. We will first implement a minimal integrated system to validate the AI pipeline, then progressively build the core data pipeline and storage model, and finally extend the system with features such as the mindmap, AI-generated insights, and sharing functionality. This approach allows us to manage risk effectively, especially for components involving AI-generated outputs.

Third, we will use Git and GitHub for version control and collaboration. We intend to maintain a structured workflow using feature branches for new functionality and clear commit messages to track changes. For more complex features, we will review each other's code before merging to ensure correctness and maintain code quality.

Fourth, we will incorporate testing at multiple levels, focusing on key parts of the system. We plan to perform unit testing on core backend components such as flashcard generation logic, metadata handling, and aggregation of topical flashcards. We will also conduct integration testing to verify that the frontend, backend, database, and AI services work correctly together as a complete pipeline. In addition, we will carry out basic user testing with peers to gather feedback on usability and the usefulness of generated notes and insights.

In addition, we will place emphasis on data modelling and structured storage design. As the system relies on transforming raw inputs into structured and interconnected knowledge, we will design our data model for coursework flashcards, topical flashcards, and metadata early in development. This ensures that later features such as the mindmap and sharing system can be built on a consistent and extensible foundation.

We will also implement a modular backend design using RESTful APIs. Key functionalities such as flashcard generation, metadata extraction, topic aggregation, AI insight generation, and note sharing will be exposed through well-defined endpoints. This improves clarity of system interactions and allows different parts of the system to be developed and tested independently.

Finally, we will maintain clear and concise documentation throughout the development process. This includes documenting the system architecture, key design decisions, database schema, and setup instructions. This will help both team members stay aligned and make it easier to extend or debug the system as it evolves.